

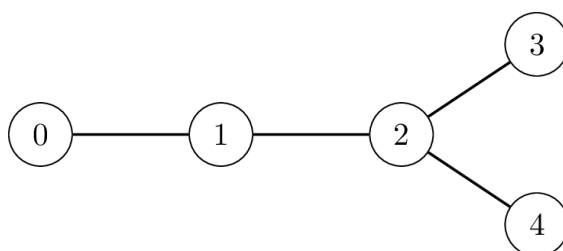
Migrazioni

Il Museo di Storia Naturale sta studiando i modelli di migrazione dei dinosauri in Bolivia. I paleontologi hanno scoperto impronte di dinosauro in N siti diversi, numerati da 0 a $N - 1$ in ordine decrescente di età: il sito 0 ha le impronte più antiche, e il sito $N - 1$ le più giovani.

I dinosauri sono migrati in ciascun sito (eccetto il sito 0) da un sito specifico più antico: per ogni sito i tale che $1 \leq i \leq N - 1$, esiste esattamente un sito più antico $P[i]$ (con $P[i] < i$) tale che alcuni dinosauri sono migrati direttamente dal sito $P[i]$ al sito i . Un singolo sito più antico può essere stato la fonte di migrazioni verso più siti più giovani.

I paleontologi modellano ogni migrazione come una **connessione non diretta** tra i siti i e $P[i]$. Si noti che per due siti distinti x e y , è possibile viaggiare da x a y attraversando una sequenza di connessioni. La **distanza** tra i siti x e y è definita come il numero minimo di connessioni necessarie per andare da x a y .

Ad esempio, l'immagine seguente mostra un caso con $N = 5$ siti, dove $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, e $P[4] = 2$. Si può viaggiare dal sito 3 al sito 4 passando per due connessioni, quindi la distanza tra i due è 2.



Il Museo mira a identificare una coppia di siti con la massima distanza possibile. Si noti che la coppia non è necessariamente unica: ad esempio, nell'esempio precedente, entrambe le coppie $(0, 3)$ e $(0, 4)$ hanno una distanza di 3, che è la massima. In questi casi, qualsiasi coppia che raggiunga la distanza massima è considerata valida.

Inizialmente, i valori di $P[i]$ **non** sono conosciuti. Il Museo invia un team di ricerca a visitare i siti $1, 2, \dots, N - 1$ in quest'ordine. Quando raggiunge ogni sito i ($1 \leq i \leq N - 1$), il team esegue entrambe le seguenti azioni:

- Scopre il valore di $P[i]$, cioè la fonte di migrazione al sito i ;
- Decide se inviare **un** messaggio al Museo o se non inviare alcun messaggio da questo sito, in base alle informazioni raccolte **fino ad ora**.

I messaggi vengono trasmessi tramite un costoso sistema di comunicazione satellitare, quindi ogni messaggio deve essere un numero intero compreso tra 1 e 20 000, incluso. Inoltre, la squadra può inviare **al massimo 50 messaggi** in totale.

Il vostro compito è quello di implementare una strategia, attraverso la quale:

- Il team di ricerca seleziona i siti da cui inviare i messaggi e il contenuto di ciascun messaggio;
- Il Museo determina una coppia di siti con la massima distanza, basandosi esclusivamente sui messaggi ricevuti da ciascun sito, sapendo da quali siti sono stati inviati.

L'invio di grandi numeri attraverso il satellite è costoso: il punteggio dipende quindi sia dal numero più grande inviato sia dal numero totale di messaggi trasmessi.

Dettagli di implementazione

È necessario implementare due funzioni: una per il team di ricerca e una per il Museo.

La funzione da implementare per il **team di ricerca** è:

```
int send_message(int N, int i, int Pi)
```

- N : il numero di siti.
- i : l'indice del sito che il team sta visitando.
- Pi : il valore di $P[i]$.
- Questa procedura viene chiamata $N - 1$ volte per ogni caso di test, nell'ordine di $i = 1, 2, \dots, N - 1$.

Questa procedura deve restituire un intero $S[i]$ che specifica l'azione intrapresa dal team di ricerca nel sito i :

- $S[i] = 0$: il team di ricerca decide di non inviare un messaggio dal sito i .
- $1 \leq S[i] \leq 20\,000$: il team di ricerca invia l'intero $S[i]$ come messaggio dal sito i .

La funzione da implementare per il **Museo** è:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : un array di lunghezza N tale che:
 - $S[0] = 0$.
 - Per ogni $1 \leq i \leq N - 1$, $S[i]$ è l'intero restituito da `send_message(N, i, Pi)`.
- Questa funzione viene richiamata esattamente una volta per ogni caso di test.

Questa funzione deve restituire una coppia di siti (U, V) con la distanza massima.

Nella valutazione in CMS, un programma che chiama le funzioni di cui sopra viene eseguito **esattamente due volte**.

- Durante la prima esecuzione del programma:
 - `send_message` viene chiamato esattamente $N - 1$ volte.
 - **Il programma è in grado di memorizzare e conservare le informazioni tra le varie chiamate successive.**
 - I valori restituiti (array S) vengono memorizzati da CMS.
 - In alcuni casi il comportamento del grader è **adattivo**. Ciò significa che il valore di $P[i]$ in una chiamata a `send_message` può dipendere dalle azioni intraprese dal gruppo di ricerca durante le chiamate precedenti.
- Durante la seconda esecuzione del programma:
 - `longest_path` viene chiamato esattamente una volta. L'unica informazione disponibile per `longest_path` dalla prima esecuzione è l'array S .

Assunzioni

- $N = 10\,000$
- $0 \leq P[i] < i$ per ogni i tale che $1 \leq i \leq N - 1$.

Subtask e punteggio

Subtask	Punteggio	Vincoli aggiuntivi
1	30	Una delle coppie con distanza massima è composta dal sito 0 con un altro sito.
2	70	Nessun vincolo aggiuntivo.

Sia Z il massimo numero intero presente nell'array S , e sia M il numero di messaggi inviati dal gruppo di ricerca.

In uno qualsiasi dei casi di test, se almeno una delle seguenti condizioni è valida, il punteggio della soluzione per quel caso di test sarà 0 (riportato come `Output isn't correct` in CMS):

- Almeno un elemento in S non è valido;
- $Z > 20\,000$ oppure $M > 50$;
- Il valore di ritorno della chiamata alla procedura `longest_path` non è corretto.

Altrimenti, il punteggio per il subtask 1 viene calcolato come segue:

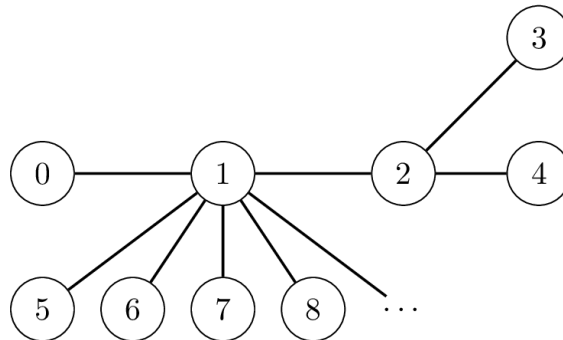
Condizione	Punteggio
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Il punteggio per il subtask 2 è calcolato come segue:

Condizione	Punteggio
$5 \leq Z \leq 20\,000$ e $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ e $32 \leq M \leq 50$	40
$Z \leq 4$ e $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ e $M \leq 8$	70

Esempio

Sia $N = 10\,000$. Si consideri la situazione in cui $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, e $P[i] = 1$ per $i > 4$.



Si supponga che la strategia del team di ricerca sia quella di inviare il messaggio $10 \cdot V + U$ ogni volta che la coppia di siti con la massima distanza cambia a seguito di una chiamata la coppia di siti (U, V) con la massima distanza cambia a seguito di una chiamata `send_message`.

Inizialmente, la coppia con la massima distanza è $(U, V) = (0, 0)$. Consideriamo la seguente sequenza di chiamate durante la prima esecuzione del programma:

Chiamata di funzione	(U, V)	Valore restituito ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Si noti che in tutte le chiamate rimanenti il valore di $P[i]$ è 1. Ciò significa che la coppia con la distanza massima non cambia, e il team non invia altri messaggi.

Quindi, nella seconda esecuzione del programma, viene effettuata la seguente chiamata:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Il Museo legge l'ultimo messaggio inviato dal gruppo di ricerca, che è $S[3] = 30$, e deduce che $(0, 3)$ è la coppia di siti con la massima distanza. Pertanto, la chiamata restituisce $(0, 3)$.

Si noti che questo approccio non consente sempre al Museo di determinare sempre correttamente la coppia con la massima distanza.

Grader di esempio

Il grader di esempio chiama sia `send_message` che `longest_path` nella stessa esecuzione, **differentemente dal grader in CMS**.

Formato dell'input:

```
N
P[1] P[2] ... P[N-1]
```

Formato di uscita:

```
S[1] S[2] ... S[N-1]
U V
```

Si noti che è possibile utilizzare il grader di esempio con qualsiasi valore di N .